🚀 Multi-User Engineering Blog Platform

Full-Stack Project Specification & Architecture Blueprint for Freshers

This implementation guide defines the software architecture, repository organization, and feature specifications required to build a production-ready, role-based engineering blog platform (Medium Clone). Developers will build a client architecture alongside a decoupled, production-grade Node.js server to gain practical experience with modern software engineering concepts.

> 💡 **Learning Objective**
>
> This project is intentionally designed with **no predefined boilerplate code**. Developers must implement all configurations, API wrappers, security layers, and asynchronous pipelines using their core engineering skills.

🛠️ 1. Professional Technology Stack

To establish modern development standards, the workspace must use the following unified ecosystem toolset:

Frontend Architecture

- **Core Framework:** **Next.js 14+** (App Router paradigm utilizing React Server Components).
- **Language Variant:** **TypeScript** (Strict compliance mode; explicit type parameters required for all states, server interfaces, and interaction inputs).
- **UI & Styling Library:** **Tailwind CSS + shadcn/ui** (Radix UI accessible primitives added directly via CLI to manage atomic interface elements).
- **Asynchronous State Layer:** **TanStack Query (React Query v5)** to orchestrate cache management, client fetches, and global loading/error interfaces.
- **Local Interface State:** **Zustand** (Ultra-lightweight state store mapping component variables and input recovery parameters).

Backend Architecture

- **Runtime Environment:** **Node.js** configured as a standalone service process isolated from the Next.js execution thread.
- **Server Framework:** **Express.js** initialized using type-safe declarations or native ECMAScript Modules.
- **Database Layer:** **MongoDB (via Mongoose)** or **PostgreSQL (via Prisma ORM)**.

📁 2. Optimized Project File Architecture

The client application must follow Next.js **Route Groups** `(...)` to divide domain rules. Ready-to-use structural components created via the shadcn CLI must be written directly within the `components/ui/` layout node:

```
frontend/
└── src/
    ├── app/
    │   ├── (feed)/                # Public discovery timeline & article render view
    │   │   ├── blog/
    │   │   │   └── [slug]/        # Dynamic route processing via unique SEO slug
    │   │   │       └── page.tsx   # Shared viewer endpoint (Open to all roles)
    │   │   └── page.tsx           # Global aggregated article landing portal
    │   ├── (auth)/                # Centered view workspace for access handlers
    │   │   ├── login/
    │   │   ├── register/
    │   │   ├── forgot-password/
    │   │   └── reset-password/
    │   ├── (creator-space)/       # Protected production console (Creators Only)
    │   │   └── dashboard/
    │   │       ├── articles/      # Personal item monitoring registry
    │   │       ├── create/        # Blank editor creation platform
    │   │       ├── edit/[id]/     # Revision editor for existing items
    │   │       ├── actions.tsx
    │   │       └── page.tsx
    │   ├── layout.tsx
    │   └── middleware.tsx         # Global role routing matrix and gatekeeper
    │
    └── components/                # Presentation architecture trees
        ├── common/                # Core functional wrappers (AuthGuard, RoleGuard)
        ├── elements/              # Dynamic block elements (RichTextEditor, ExportPDF)
        ├── layout/                # Shared structural landmarks (Navbar, CreatorSidebar)
        └── ui/                    # Centralized shadcn UI primitives
            ├── button.tsx
            ├── skeleton.tsx       # Animated placeholders for loading states
            ├── dialog.tsx         # Overlay verification prompts (Destructive actions)
            ├── dropdown-menu.tsx
            ├── input.tsx
            ├── select.tsx
            └── switch.tsx         # Toggle controls for state transitions (Draft/Publish)
```

🎯 3. Mandatory Functional Feature Specifications

3.1 Three-Way Authentication & Dynamic Role Routing

Implement a modern authentication flow using **Auth.js (NextAuth v5)** that hooks into your custom Node.js backend. The platform requires three independent login vectors and enforces two strict roles:

- **Three Sign-In Methods:** Standard Email/Password registration check, Social OAuth Redirect via Google Identity, and an alternate OAuth Pop-up window interface.
- **Role-Based Redirect Targets:**
  - `VISITOR` : Allowed to view public layouts and read full articles. Upon login, they are redirected automatically to the **Global Blog Feed** (`/`). They are locked out of the dashboard.
  - `CREATOR` : Full administrative privileges. Upon login, they are directed to the **Creator Dashboard Inventory** (`/dashboard/articles`).
- **Protected Guards:** Create a global Next.js middleware file coupled with a `RoleGuard.tsx` component. If an account with a `VISITOR` role attempts to access the `/dashboard` path, intercept the routing execution and throw a `403 Access Denied` layout block.

3.2 Dynamic Routing & Search Engine Optimization (SEO)

- **SEO-Friendly Permalinks:** URL parameters must never reveal database keys or raw object IDs (e.g., avoid `/blog/65f2100bc12...`). The details router must parse readable alphanumeric text strings instead (e.g., `/blog/nextjs-state-management-best-practices`).
- **Dynamic Meta Injection:** Export the native Next.js `generateMetadata` handler from within the dynamic `[slug]/page.tsx` component file. This layer must request post details from the Node.js API, then inject custom title tags, description headers, and search criteria directly into the document header block before page delivery.

3.3 Post Workstation Layout & Form Structure

The creation workspace must build forms using shadcn UI elements, mapping validation patterns cleanly. The UI must collect **6 mandatory inputs**:

1. **Title Input:** Standard text field (`shadcn/ui Input`) that serves as the visual header and automatically creates a fallback URL string slug.
2. **Category Picker:** A structured select list (`shadcn/ui Select`) to classify the post under specific searchable indexes.
3. **Featured Cover Image:** An asset selector or text URL reference linking the cover graphic layout.
4. **Short Excerpt/Summary:** A concise text summary box that serves as the listing description and fills the page's meta metadata tags.
5. **SEO Keywords Input:** A dedicated string field allowing creators to append comma-separated tags to optimize dynamic search discovery headers.

6. **Rich Content Workspace:** Integrate a robust **HTML Rich Text Editor** (such as *TipTap*, *Quill*, or *Editor.js*) instead of a native text box. This ensures users can insert headings, lists, and code blocks while exporting clean HTML directly to the database.

3.4 Content Life Cycle & Workspace Management

Creators manage their articles within the protected `/dashboard/articles` module via explicit CRUD interactions:

- **Create & Update:** Interface buttons linked to workspace form parameters.
- **Delete Controller:** Destructive commands must be wrapped inside an interactive `shadcn/ui Dialog` alert to request confirmation before executing API deletions.
- **Live Status Toggle:** Mount a quick switch tool (`shadcn/ui Switch`) directly within the inventory listing table row. This allows creators to instantly transition an article's visibility status between `DRAFT` and `PUBLISHED` without opening the editor workspace.

> 🔒 **State Segregation Pattern**
>
> **Server State (TanStack Query):** Handles server-side items, listing matrices, and mutations. Configured with strict validation lifetimes to manage automatic data updates and cache validation routines.
>
> **Client UI State (Zustand):** Manages ephemeral variables like layout modes and drawer states. Includes a **Persistent Form Cache** that saves editor inputs locally. If a user closes their browser window or navigates away unexpectedly, their unsaved content reloads automatically when they return.

3.5 Visual Loading Skeletons

To prevent jarring layout shifts during data fetching, designers must build animated placeholders inside `components/ui/skeleton.tsx`:

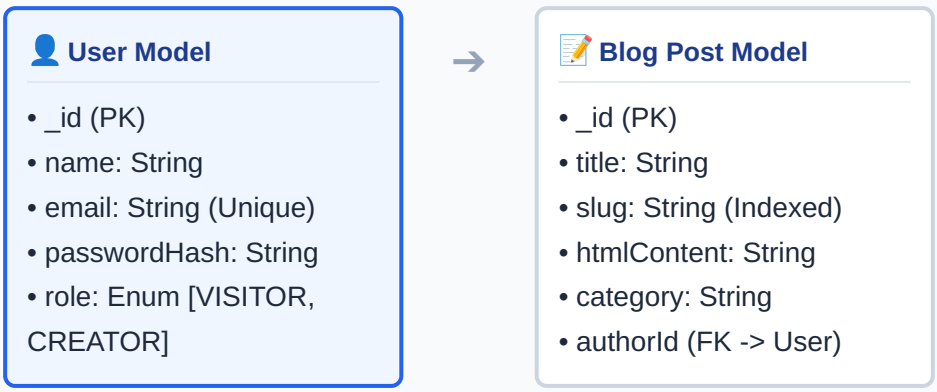
- **Listing Skeleton:** Renders a grid of cards with animated blocks that match the text lines, image frames, and margins of the active feed view.
- **Details Page Skeleton:** Displays a structured placeholder layout mapping the primary heading line, author avatar block, and text layout rows until the backend payload resolves.

3.6 Declarative Document Print Engine (PDF Export)

- Add a **"Download PDF Copy"** action button to the article detail view.
- When clicked, the client layer compiles the article's data into a document using `@react-pdf/renderer` inside an isolated client thread.
- The output document must feature a professional layout with a **custom recurring header** (containing the platform name and article title) and a **custom recurring footer** tracking page numbers (e.g., "Page X of Y"), rendering cleanly across page breaks.

💾 4. Backend Database Schemas (Node.js & Express)

The backend application must run independently of Next.js and maintain clean database relationships. The data models must follow these structures:



User Table / Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId / UUID	Primary Key	Unique record index identifier.
name	String	Required	The visible display name of the account profile.
email	String	Unique, Indexed	The identifier target used for login security.
passwordHash	String	Required (Conditional)	Encrypted credential string (null for pure OAuth accounts).
role	Enum / String	Default: `"VISITOR"`	Role privileges: `"VISITOR"` or `"CREATOR"`.
createdAt	Timestamp	Auto-generated	Chronological record tracker for user onboarding.

Blog Post Table / Schema

Field Name	Data Type	Constraints	Description
_id	ObjectId / UUID	Primary Key	Unique record index identifier.
title	String	Required	The main title header text of the article.
slug	String	Unique, Indexed	Clean, URL-safe text reference string for SEO optimization.
htmlContent	String	Required	Raw layout markup text exported from the text editor.
category	String	Required	The indexing tag used for classification filters.
coverImage	String	Optional	The network URL path pointing to a thumbnail header graphic.
excerpt	String	Required	A short summary sentence used for summary listing spaces.
seoKeywords	String	Optional	Comma-separated search engine indexing strings.
status	Enum / String	Default: `"DRAFT"`	Lifecycle step configuration: `"DRAFT"` or `"PUBLISHED"`.
authorId	ObjectId / UUID	Foreign Key	Relational link mapping directly back to the User `_id`.
createdAt	Timestamp	Auto-generated	Chronological record tracker for content updates.

Controller Security & Middleware Enforcement

- **Password Cryptography:** Plain-text passwords must never be stored directly. Encrypt credentials using secure hashing functions (such as `bcrypt` or `argon2`) before database persistence.
- **Role Middleware:** Create an Express verification interceptor (e.g., `checkRole(["CREATOR"])`) to protect modification routes. If a user authenticated with a `VISITOR` token calls `POST /api/posts`, the server must automatically terminate execution and throw a `403 Forbidden` error response.
- **Data Ownership Verification:** For write operations (such as `PUT /api/posts/:id` or `DELETE /api/posts/:id`), verify that the incoming token's payload ID matches the target post's `authorId` exactly before modifying any record.

🏁 5. Step-by-Step Implementation Phases

To execute this engineering project cleanly, complete development milestones in the following order:

1. **Phase 1 — Isolated Server Architecture:** Build the Node.js application container. Connect your database instances, define data schemas, code password encryption routines, issue role-infused JWT keys, and thoroughly test access controllers via an explicit API tester (e.g., Postman, Bruno).
2. **Phase 2 — Workspace Layout & Auth Matrix:** Initialize the Next.js frontend directory structure. Scaffold the route group paths and install necessary shadcn UI component primitives. Configure Auth.js controllers to link credentials and social login vectors, then apply the role-based middleware redirection guard.
3. **Phase 3 — Asynchronous Discovery Feed & SEO:** Build the public discovery timeline and dynamic details page (`/blog/[slug]`). Implement TanStack Query functions to channel server calls, structure multi-tiered skeleton components to handle loading flags gracefully, and implement the dynamic `generateMetadata` parameters.
4. **Phase 4 — Production Studio & Local Store Cache:** Build the internal creator dashboard console. Map out your dashboard item list with row-level toggle switches and deletion confirmation alerts, mount the Rich Text HTML Editor workspace, and use Zustand to persist and recover unsaved editor content.
5. **Phase 5 — PDF Generation & Document Testing:** Embed the `@react-pdf/renderer` engine to compile public article metrics into a background download stream. Set up professional margin spaces alongside recurring header and footer pagination markers, clean up layout shifting anomalies, and ensure comprehensive type safety across all execution layers.